

Team 3

Edge Intelligence

Topic : - Netflix Streaming with Edge Log Analytics

Steps:

Step 1: Synthetic Data Generation

You created realistic streaming logs using:

- faker → user/session IDs
- random → simulate behavior
- datetime → timestamps

✓ What you simulated:

- User sessions (normal / binge / dropoff)
- Playback events (play, pause, seek, stop)
- Network conditions (latency, buffering, rebuffer)
- Errors (network failure, crash)
- Devices + resolution

Output:

 netflix_logs_1000.json (or 50,000 records)

Step 2: Load Data for Analysis

```
df = pd.DataFrame(data)
```

Converted JSON logs → Pandas DataFrame for analytics.

Step 3: Buffering Analysis

```
df["buffer_issue"] = ...
```

✓ Logic:

- If buffering > 2000 ms → **High buffering**
- Else → **Normal**
- Helps detect **poor streaming experience**

Step 4: Network Quality Detection

```
df["network_issue"] = ...
```

✓ Logic:

- Latency > 150 ms → **Low quality stream**
- Else → **Good**

👉 Identifies **network-related issues**

Step 5: Error Detection

```
df["error_issue"] = ...
```

✓ Logic:

- If error exists → **Playback failure**

👉 Helps detect:

- App crashes
- Streaming failures

Step 6: CDN + Region Simulation

```
df["cdn"] = random.choice(...)
```

```
df["region"] = random.choice(...)
```

✓ CDNs simulated:

- Akamai
- CloudFront
- OpenConnect

Analysis:

```
df.groupby("cdn")["buffering_ms"].mean()  
df.groupby("region")["buffering_ms"].mean()
```

👉 Helps identify:

- Slow CDN

- Problematic regions

Step 7: Decision Engine (Key Feature 🌟)

```
def decide_action(row):
```

✓ Intelligent actions:

- High buffering → Reduce bitrate
- Error → Switch CDN
- High latency → Adjust quality
- Else → No action

👉 This is your **core system intelligence**

Step 8: Aggregated Insights

```
avg_buffer = ...
```

```
error_rate = ...
```

You computed:

- Average buffering
- Average watch time
- Error rate

👉 Used for **overall system health**

Step 9: Visualization

Used matplotlib:

- ✓ Buffering distribution
- ✓ Error frequency
- ✓ Watch-time trend

👉 Helps understand patterns visually

Step 10: Clustering + t-SNE

TSNE(n_components=2)

✓ Reduced high-dimensional data → 2D

✓ Visualized clusters

👉 Helps identify:

- Good sessions
- Problem sessions
- Critical failures

Step 11: Cluster Analysis

```
df.groupby("cluster")[["buffering_ms","latency_ms"]]  
df["cluster"].value_counts()
```

👉 You categorized user experience into clusters like:

- Smooth streaming
- Moderate issues
- Severe issues

```

17]: import json
import random
from datetime import datetime, timedelta
from faker import Faker

fake = Faker()

EVENT_TYPES = ["play", "pause", "stop", "seek"]
DEVICES = ["mobile", "tv", "web"]
RESOLUTIONS = ["480p", "720p", "1080p", "4K"]
NETWORK_TYPES = ["WiFi", "4G", "5G"]
ERROR_CODES = [None, "BUFFER_TIMEOUT", "NETWORK_ERROR", "PLAYER_CRASH"]

TARGET_RECORDS = 50000 # total Logs required

def random_time_gap():
    return random.uniform(1, 60)

def simulate_network():
    base_latency = random.randint(20, 80)
    if random.random() < 0.1:
        return base_latency + random.randint(100, 500), random.randint(500, 2000), random.randint(1, 5)
    return base_latency, random.randint(0, 300), random.randint(0, 1)

def simulate_error():
    if random.random() < 0.1:
        return random.choice(ERROR_CODES[1:]), True
    return None, False

```

```

def user_behavior():
    r = random.random()
    if r < 0.5:
        return "normal"
    elif r < 0.8:
        return "binge"
    return "dropoff"

def generate_session(user_id):
    session_id = f"sess_{fake.uuid4()}"
    behavior = user_behavior()

    num_events = {
        "normal": random.randint(5, 15),
        "binge": random.randint(20, 40),
        "dropoff": random.randint(2, 5)
    }[behavior]

    base_time = datetime.utcnow()
    watch_time = 0
    logs = []

    for i in range(num_events):
        latency, buffering, rebuffer = simulate_network()
        error, crash = simulate_error()

        watch_time += random.randint(10, 120)

```

```

log = {
    "event_id": fake.uuid4(),
    "timestamp": (base_time + timedelta(seconds=i * random_time_gap())).isoformat(),
    "user_id": user_id,
    "session_id": session_id,
    "event_type": random.choice(EVENT_TYPES),

    "content_id": f"cnt_{random.randint(1,100)}",
    "device_type": random.choice(DEVICES),
    "network_type": random.choice(NETWORK_TYPES),

    "watch_time_sec": watch_time,
    "latency_ms": latency,
    "buffering_ms": buffering,
    "rebuffer_count": rebuffer,
    "resolution": random.choice(RESOLUTIONS),

    "error_code": error,
    "crash_flag": crash,
    "retry_attempts": random.randint(1,3) if error else 0,

    "behavior": behavior
}

logs.append(log)

return logs

# -----
# Generate ~50000 records
# -----
all_logs = []
users = [f"user_{i}" for i in range(50)]

```

```

while len(all_logs) < TARGET_RECORDS:
    user = random.choice(users)
    session_logs = generate_session(user)
    all_logs.extend(session_logs)

# Trim exactly 1000
all_logs = all_logs[:TARGET_RECORDS]

# Save to JSON file
with open("netflix_logs_50000.json", "w") as f:
    json.dump(all_logs, f, indent=2)

print(f"Generated {len(all_logs)} records successfully!")

```

Generated 50000 records successfully!

```

[19]: import json
import pandas as pd

# -----
# Load Data
# -----
with open("netflix_logs_50000.json", "r") as f:
    data = json.load(f)

df = pd.DataFrame(data)

```

```

[20]: # -----
# Step 1: Buffering Analysis
# -----
df["buffer_issue"] = df["buffering_ms"].apply(
    lambda x: "High buffering" if x > 2000 else "Normal"
)

# -----
# Step 2: Network Quality Detection
# -----
df["network_issue"] = df["latency_ms"].apply(
    lambda x: "Low quality stream" if x > 150 else "Good"
)

# -----
# Step 3: Error Detection
# -----
df["error_issue"] = df["error_code"].apply(
    lambda x: "Playback failure" if pd.notnull(x) else "No error"
)

```

```
[21]: # -----
# Step 4: CDN Performance
# (simulate CDN column if missing)
# -----
import random
cdns = ["Akamai", "CloudFront", "OpenConnect"]

if "cdn" not in df.columns:
    df["cdn"] = [random.choice(cdns) for _ in range(len(df))]

# Slow CDN (based on avg buffering)
cdn_performance = df.groupby("cdn")["buffering_ms"].mean().sort_values(ascending=False)

# Region issues (simulate region if missing)
if "region" not in df.columns:
    df["region"] = [random.choice(["India", "US", "EU"]) for _ in range(len(df))]

region_performance = df.groupby("region")["buffering_ms"].mean().sort_values(ascending=False)

[22]: # -----
# Step 5: Decision Making
# -----
def decide_action(row):
    if row["buffering_ms"] > 2000:
        return "Reduce bitrate"
    elif pd.notnull(row["error_code"]):
        return "Switch CDN"
    elif row["latency_ms"] > 150:
        return "Adjust quality"
    else:
        return "No action"

df["action"] = df.apply(decide_action, axis=1)

# -----
# Step 6: Aggregation (Insights)
# -----
avg_buffer = df["buffering_ms"].mean()
avg_bitrate = df["watch_time_sec"].mean() # proxy (since bitrate not present)
error_rate = df["error_code"].notnull().mean()

print("\n📊 Aggregated Metrics:")
print("Average Buffering (ms):", avg_buffer)
print("Average Watch Time:", avg_bitrate)
print("Error Rate:", error_rate)

print("\n📁 CDN Performance (Worst to Best):")
print(cdn_performance)

print("\n🌐 Region Performance (Worst to Best):")
print(region_performance)
```

📊 Aggregated Metrics:

Average Buffering (ms): 260.3542

Average Watch Time: 776.86044

Error Rate: 0.09854

📁 CDN Performance (Worst to Best):

cdn

OpenConnect 261.742835

Akamai 261.715328

CloudFront 257.596898

Name: buffering_ms, dtype: float64

🌐 Region Performance (Worst to Best):

region

US 261.106894

EU 260.995275

India 258.959659

Name: buffering_ms, dtype: float64

```

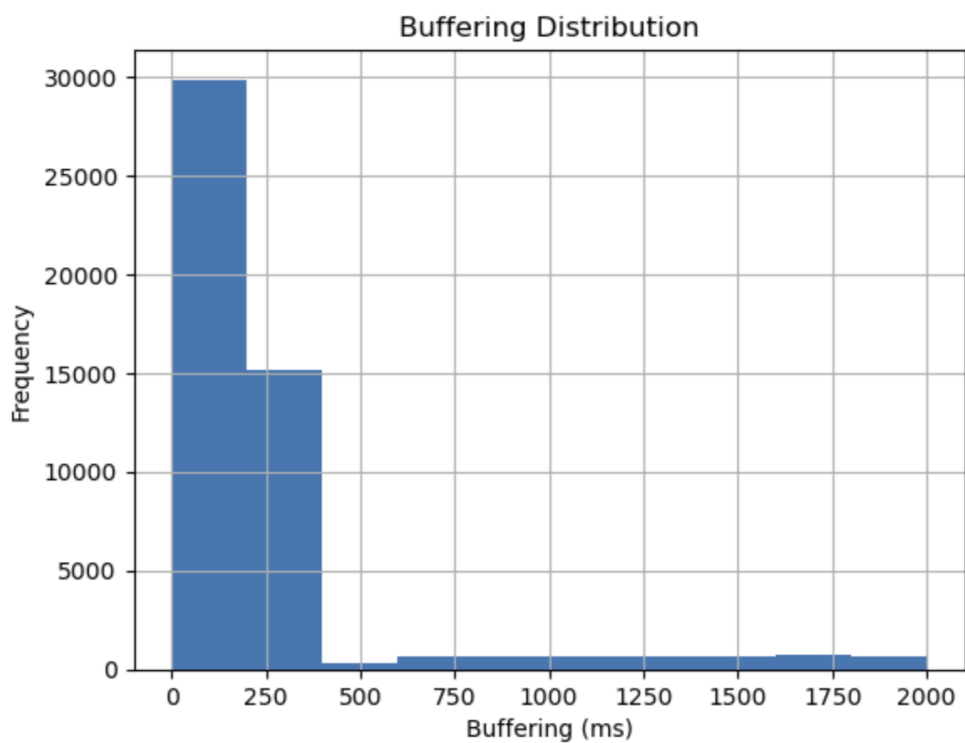
# -----
# Step 7: Visualization
# -----
import matplotlib.pyplot as plt

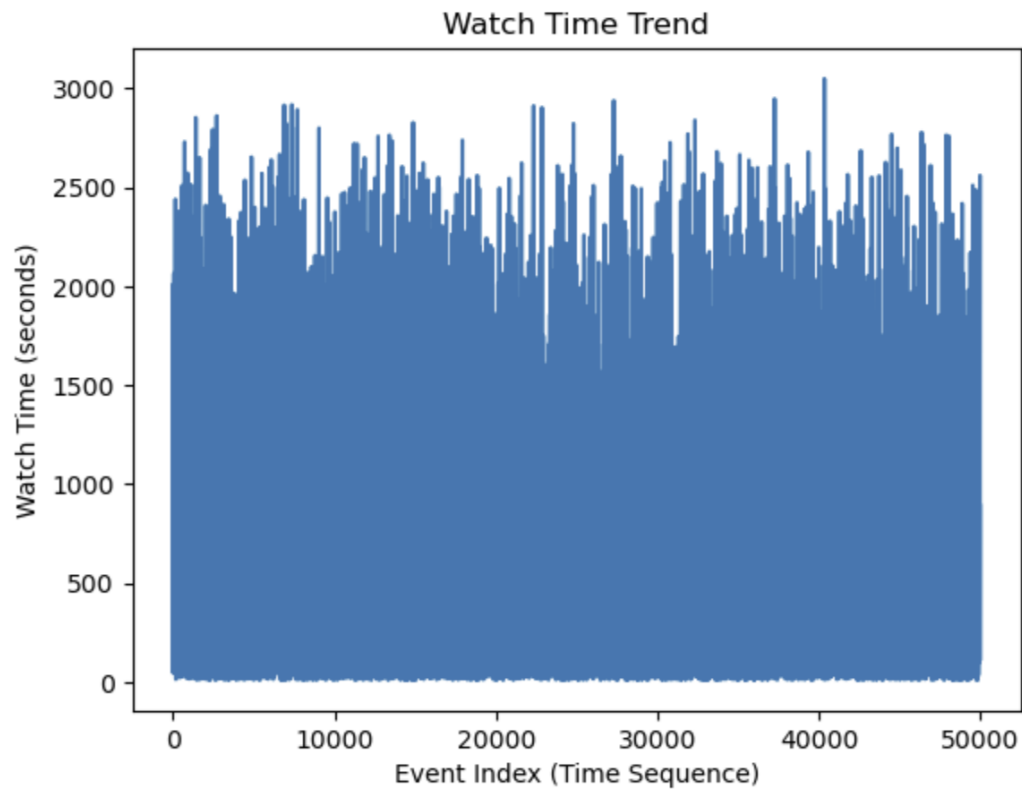
# Buffering Distribution
plt.figure()
df["buffering_ms"].hist()
plt.title("Buffering Distribution")
plt.xlabel("Buffering (ms)")
plt.ylabel("Frequency")
plt.show()

# Error Frequency
plt.figure()
df["error_issue"].value_counts().plot(kind="bar")
plt.title("Error Frequency")
plt.show()

# Bitrate Proxy Trend (watch time)
plt.figure()
df["watch_time_sec"].plot()
plt.title("Watch Time Trend")
plt.show()

```





```
[33]: # -----  
# Preview Results  
# -----  
print("\nSample Output:")  
print(df[[  
    "buffering_ms",  
    "latency_ms",  
    "error_code",  
    "buffer_issue",  
    "network_issue",  
    "error_issue",  
    "action"  
]].head())
```

Sample Output:

	buffering_ms	latency_ms	error_code	buffer_issue	network_issue	\
0	69	77	None	Normal	Good	
1	1644	253	None	Normal	Low quality stream	
2	814	357	None	Normal	Low quality stream	
3	245	79	None	Normal	Good	
4	133	55	None	Normal	Good	

	error_issue	action
0	No error	No action
1	No error	Adjust quality
2	No error	Adjust quality
3	No error	No action
4	No error	No action

```
[38]: from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

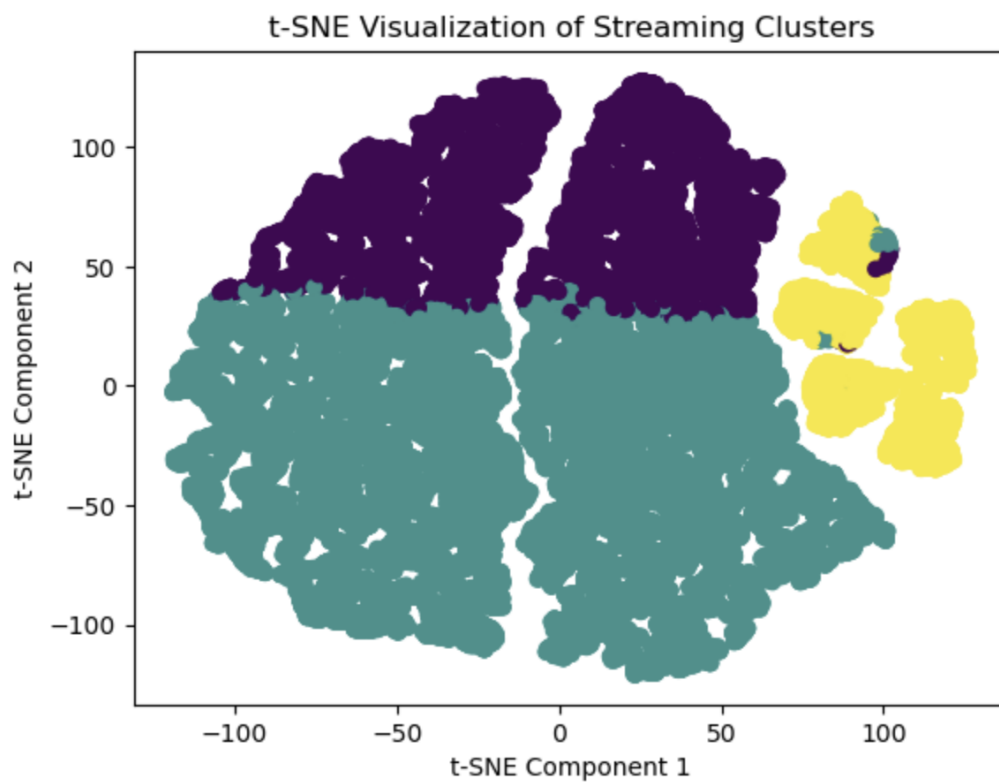
# -----
# Apply t-SNE (reduce to 2D)
# -----
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
tsne_result = tsne.fit_transform(scaled_features)

# -----
# Plot t-SNE clusters
# -----
plt.figure()

plt.scatter(
    tsne_result[:, 0],
    tsne_result[:, 1],
    c=df["cluster"]
)

plt.xlabel("t-SNE Component 1")
plt.ylabel("t-SNE Component 2")
plt.title("t-SNE Visualization of Streaming Clusters")

plt.show()
```



```
[40]: df.groupby("cluster")[["buffering_ms", "latency_ms"]].mean()
```

```
[40]:
```

	buffering_ms	latency_ms
cluster		
0	150.442968	50.342122
1	151.742494	50.604310
2	1261.080123	357.193238

```
[41]: df["cluster"].value_counts()
```

```
[41]: cluster
1      31277
0      13843
2       4880
Name: count, dtype: int64
```

What is the MAIN FEATURE?

👉 Your system's **main feature** is:

★ **Real-Time Streaming Quality Optimization**

It:

- Monitors logs
- Detects issues
- Makes automatic decisions

What is the dataset about?

- Synthetic dataset simulating **Netflix streaming logs**
- Represents **user playback behavior + network performance**
- Generated to mimic **real-time CDN edge analytics**

What attributes are present?

👤 **User & Session**

- user_id
- session_id

Playback Info

- event_type
- content_id
- watch_time_sec

Device & Network

- device_type
- network_type
- resolution

Performance Metrics

- latency_ms
- buffering_ms
- rebuffer_count

Error Tracking

- error_code
- crash_flag
- retry_attempts

Behavior

- behavior (normal / binge / dropoff)

Added During Analysis

- buffer_issue
- network_issue
- error_issue
- cdn
- region
- action
- cluster

What problem statement are you solving?

Problem:

- Users face buffering, latency, and playback failures during streaming.

Solution:

Build a system that:

- Detects streaming issues in real time
- Identifies root causes (network/CDN/errors)
- Automatically suggests corrective actions



Final Goal:



Improve:

- Video quality
- User experience
- Streaming reliability



Final Summary



“We built a Netflix-like edge log analytics system that processes streaming logs, detects buffering and network issues, and dynamically recommends actions like bitrate adjustment or CDN switching to improve user experience.”